

# REDELf: An Energy-Efficient Deadlock-Free Routing for 3D NoCs with Partial Vertical Connections

JINHO LEE, Seoul National University  
KYUNGSU KANG, Samsung  
KIYOUNG CHOI, Seoul National University

3D *integrated circuits* (3D ICs) using *through-silicon vias* (TSVs) allow to envision the stacking of dies with different functions and technologies, using as an interconnect backbone a 3D *network-on-chip* (NoC). However, partial vertical connection in 3D NoCs seems unavoidable because of the large overhead of TSV itself (e.g., large footprint, low fabrication yield, additional fabrication processes) as well as the heterogeneity in dimension. This article proposes an energy-efficient deadlock-free routing algorithm for 3D mesh topologies where vertical connections partially exist. By introducing some rules for selecting elevators (i.e., vertical links between dies), the routing algorithm can eliminate the dedicated virtual channel requirement. In this article, the rules themselves as well as the proof of deadlock freedom are given. By eliminating the virtual channels for deadlock avoidance, the proposed routing algorithm reduces the energy consumption by 38.9% compared to a conventional routing algorithm. When the virtual channel is used for reducing the head-of-line blocking, the proposed routing algorithm increases performance by up to 23.1% and 6.9% on average.

Categories and Subject Descriptors: C.2.2 [Computer-Communication Networks]: Network Protocols

General Terms: Design, Algorithms, Performance

Additional Key Words and Phrases: NoC, deadlock, routing algorithm, 3D stacking

## ACM Reference Format:

Jinho Lee, Kyungsu Kang, and Kiyong Choi. 2015. REDELf: An energy-efficient deadlock-free routing for 3D NoCs with partial vertical connections. *ACM J. Emerg. Technol. Comput. Syst.* 12, 3, Article 26 (September 2015), 22 pages.

DOI: <http://dx.doi.org/10.1145/2751560>

## 1. INTRODUCTION

As more transistors tend to be integrated in a chip, the many-core *chip multiprocessor* (CMP) equipped with an NoC has rapidly become a research trend. One of the issues in NoC-related research is how to make an efficient NoC with the advanced 3D stacking technologies, since 3D stacking provides not only more silicon area, but also opportunities for higher bandwidth and shorter wire lengths by using the additional vertical connections of *through-silicon vias* (TSVs) [Loh 2008]. In addition, 3D stacking enables stacking dies of disparate technologies (e.g., DRAM, CMOS, MEMS, RF) together in a single package.

Even though 3D NoCs introduce a whole new set of application possibilities, there are some challenges in the use of TSVs. Each TSV requires a pitch of tens of micrometers, which consumes silicon area as well as the area on all metal layers. Also, an

---

This work was supported by the National Research Foundation of Korea (NRF) grants funded by the Korean government (MEST) (no. 2012R1A2A2A06047297).

Authors' addresses: J. Lee, Seoul National University, 1 Gwanak-ro, Gwanak-gu, Seoul, South Korea; K. Kang, Samsung, 95, Samsung 2-ro, Giheung-gu, Yongin-si, Gyeonggi-do, Korea; K. Choi (corresponding author), Seoul National University, 1 Gwanak-ro, Gwanak-gu, Seoul, South Korea; email: [kchoi@snu.ac.kr](mailto:kchoi@snu.ac.kr).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2015 ACM 1550-4832/2015/09-ART26 \$15.00

DOI: <http://dx.doi.org/10.1145/2751560>

implementation of TSV includes additional complex fabrication processes, resulting in mechanical stress on silicon and difficulty in testing before stacking. Thus, the use of large numbers of TSVs will increase fabrication cost due to the increased silicon area, the additional fabrication processes, and the reduced fabrication yield [Kang et al. 2014]. Furthermore, the low yield of TSV itself may cause partial disconnections of vertical links [Loi et al. 2011]. For these reasons, 3D NoCs tend to use only partially connected vertical links. The flexibility of stacking dies of different technologies also makes 3D NoCs incapable of keeping a homogeneous topology (the same size and shape) for all the stacked dies and using the regular fully connected 3D cube topology.

For a deadlock-free routing algorithm of 3D NoCs with partially connected vertical links, Dubois et al. [2013] first introduced an elevator-first algorithm for an architecture that connects layers of 2D NoCs with arbitrary partial connections of TSV links. It was proven that, with any network topology and any deadlock-free 2D routing algorithm, a deadlock-free 3D routing algorithm could be built using two dedicated *virtual channels* (VCs). In addition to the proof, the authors applied the elevator-first algorithm to a set of meshes using XY routing to show its effectiveness. Bahmani et al. [2012] described the detailed architecture and the cost analysis of the elevator-first algorithm. In addition, based on the algorithm, Foroutan et al. [2014] proposed a heuristic scheme of the assignment of elevators (i.e., vertical links) for maximizing the performance of 3D NoCs. However, considering that FIFO buffers are the second largest component of NoC power consumption [Kahng et al. 2009], the dedicated VCs used in the elevator-first algorithm become the main burden for energy efficiency (i.e., energy/performance). Also, dedicating VCs to deadlock freedom often fails in balancing loads between the VCs, and thus shows inferior performance compared to other approaches using VCs to avoid HoL (*head-of-line*) blockings.

In this article, we propose an energy-efficient deadlock-free routing algorithm for 3D NoCs where planar deadlock-free networks are partially connected by vertical links. We focus on mesh topology for the planar networks, which is the most popular among a wide variety of NoC topologies. This comes from its simplicity, scalability, intuitiveness, and easy placement of tiles and wires. Much existing NoC research has used mesh for topology [Fallin et al. 2012; Hoskote et al. 2007; Tiler 2009]. We use dimension-ordered (i.e., XY) routing, which was also adopted in Dubois et al. [2013]. The contributions of this article are as follows.

- (1) Through close examination of the layered mesh topology, we prove that the proposed algorithm can be made deadlock free, even without the dedicated VCs for vertical links, while maintaining the benefits of the elevator-first routing algorithm: having flexibility that allows random vertical connections and not requiring the large lookup tables needed in many topology-agnostic routing algorithms.
- (2) In order to see the effects of VCs on energy efficiency and performance, the proposed algorithm is evaluated in terms of application latency, energy, and energy-delay product compared to the elevator-first algorithm for 3D NoCs with different numbers of vertical links connected. The experimental results are measured using NoC performance and a power simulator with both synthetic and real benchmark programs.

The rest of this article is composed as follows. Section 2 shows related work and Section 3 explains the baseline elevator-first algorithm. Section 4 explains our algorithm with proofs, Section 5 shows experimental results, and Section 6 concludes the article.

## 2. RELATED WORK

General-purpose packet-switched 3D NoCs, which deliver packet between processing element such as IP blocks, processors, DSP cores, memory blocks, FPGA blocks, etc.,

have been extensively studied for large 3D systems-on-chip. In particular, there has been a large body of work that focuses on extending a 2D mesh topology to 3D NoCs. Li et al. [2006] proposed a hybridization of a bus structure and a 2D mesh network, employing the bus structure for communications among processing elements located on different layers. Furthermore, in conjunction with hybrid bus-mesh interconnect, Kim et al. [2007] proposed a dimensionally decomposed crossbar design that reduces packet contention in routers, and Rahmani et al. [2011] proposed a congestion-aware adaptive routing algorithm for inter-layer communications. Dahir et al. [2012] extended the odd-even turn model for 3D meshes. Darve et al. [2011] proposed a design of 3D NoC router that has seven ports (one port to the IP block, one each to routers above and below, and one in each cardinal direction) in order to implement a true 3D mesh (i.e., 3D cube) network. However, these differ from our target platform in that they do not consider the limited connectivity along vertical directions.

Because of the large overhead of TSV itself, several attempts were made to reduce the number of TSVs in 3D NoCs. Pasricha [2009] proposed serialization of TSVs for area reduction, and also studied the synthesis of 3D NoCs considering TSV serialization [Pasricha 2012]. Zhu et al. [2012] suggested an adaptive routing algorithm for a 3D mesh with limited number of TSV links. Lee et al. [2013a, 2013b] used a kind of deflection routing to alleviate the congestion on the vertical links. Akesson et al. [2011] and Liu et al. [2011] proposed asymmetric mesh topologies, where only a few 3D NoC routers support inter-tier communications while the rest are allowed only horizontal communications. Such topologies can also reduce the number of vertical links, sacrificing bandwidth of inter-tier communications. In Xu et al. [2010], the optimal number of TSVs is analyzed to reduce the chip footprint, and thus the network latency, while sacrificing bandwidth of inter-tier communications. In Kumar et al. [2011], an algorithm for 3D layer assignment of NoC components was proposed in order to reduce the number of vertical links.

For reducing power consumption of 3D NoCs, Xu et al. [2009] proposed a low-diameter 3D interconnect network using low-radix routers. The reduced hop counts through the 3D network consume less power. Park et al. [2008] have looked at decomposing a NoC router into third dimension that helps reduce the chip footprint and power consumption. In addition, there has been research on designing custom 3D NoC topologies focusing on power efficiency improvement [Kapadia and Pasricha 2012; Seiculescu et al. 2009; Yan and Lin 2011].

There has been research on topology-agnostic deadlock-free routing for arbitrary networks. Up/down routing [Schroeder et al. 1991] forms a spanning tree from the network and assigns each channel up direction or down direction. Deadlock is avoided by always taking up-directed channels before taking any down-directed channels. Such routing can be applied to networks that allow random connections, as in wireless NoCs [Matsutani et al. 2013; Wettin et al. 2014]. Up/down routing gained popularity because of its simplicity, and many variations came out for performance [Koibuchi et al. 2001; Sancho et al. 2000a, 2000b], but they are not scalable because of the full lookup tables needed in each router. Borkar et al. [1988] and Wu and Sheng [1999] attempted to reduce the lookup table size. Segment-based routing [Mejia et al. 2006] divides planar meshes or tori into subnets and uses heuristics to prohibit turns within each subnet and guarantee global deadlock freedom. LASH [Skeie et al. 2002], ALASH [Lysne et al. 2006], and MROOTS [Lysne et al. 2006] also use a deadlock-free algorithm for irregular networks, but it relies on using VCs. FDOR [Skeie et al. 2009] is another deadlock-free routing for 3D irregular networks, but assumes the vertical links are fully connected. There have been many other attempts for simple and high-performance deadlock-free algorithms on irregular networks, which were summarized in Flich et al. [2012].

Although the aforementioned topology-agnostic routing algorithms can be used for the partially connected 3D mesh, they are far too general. Even though the vertical channels of partially connected 3D meshes can be placed randomly, the 2D network layers are planar meshes which have uniform patterns. The topology-agnostic routings do not benefit much from such uniformity. Also, they often suffer unbalanced traffic load because they usually utilize a spanning tree to generate the algorithm. Furthermore, the irregular routing path leads to complex routing table requirements. Our algorithm greatly reduces such overhead by simply assigning elevators to each node and using the existing algorithm within planar networks to reach them.

---

**ALGORITHM 1:** Pseudocode of Elevator-First Algorithm

---

**Elevator-first routing algorithm (src, current, dest)**

```

//assign virtual channel
if (current node is the source node) {
    if (src.z > dest.z) {
        assign the packet to "up" vc;
    } else if (src.z < dest.z) {
        assign the packet to "down" vc;
    } else {
        assign the packet to any vc;
    }
}
//route
if (current.z == dest.z) {
    route to dest using 2D routing algorithm (XY);
} else {
    if (current node is an elevator node) {
        route towards dest.z thru the elevator;
    } else {
        route to an elevator node using 2D
        routing algorithm (XY);
    }
}

```

---

### 3. BACKGROUND

Our algorithm is based on the elevator-first algorithm and thus it is necessary to explain it prior to our algorithm. The elevator-first algorithm has been proposed as one that connects several layers of 2D networks that use deadlock-free algorithms. The key point of the algorithm is that it is deadlock free even though there are only partial, arbitrary vertical connections, while only two virtual channels are needed.

A simplified pseudocode of the elevator-first algorithm is shown in Algorithm 1. When a packet is destined to a node located on another layer, it is first routed to a vertical link (an elevator) using the routing algorithm inside the 2D network (e.g., XY routing). When it arrives at the elevator, the packet is sent upward or downward (towards the destination). When the packet reaches the next layer, the destination is checked. If the destination is on the current layer, then it is routed to the destination using the routing algorithm of the new layer. If it is not, the same process is repeated until it reaches the layer of the destination node.

The routing itself is not deadlock free without virtual channels, but with two virtual channels only inside the 2D networks, it can be made deadlock free. The key to ensuring deadlock freedom is to separate the virtual channels into upward and downward

channels. When a packet is to be routed to an upper layer, it only uses upward virtual channels and upward vertical links. When the destination is on a lower layer, it uses downward virtual channels and downward vertical links. This eliminates cycles from the channel dependency graph and deadlock freedom is ensured. For a detailed proof, refer to Dubois et al. [2013].

It is also worth mentioning that the elevator-first algorithm requires almost no space for the routing table. Assuming that each node is assigned a single fixed elevator for both (up/down) directions, it only has to store locations for the two elevators<sup>1</sup>. If each layer in the network consists of  $M$  nodes, it is only  $2\log_2(M)$  bits per router. On the other hand, for topology-agnostic algorithms which demand full routing tables, each router should store the sequence of node ids through the path for all possible destinations. In such a case, the size of the table would be  $(N-1) \cdot D \log_2(N)$  bits per router, where  $N$  is the number of nodes in the network and  $D$  is the maximum distance of the routing path. If the maximum degree of the routers is limited, the port number can be used instead of node id. Still, the routing table size would be  $(N-1) \cdot D \log_2(P)$  bits per router, where  $P$  is the number of ports. For a  $4 \times 4 \times 4$  network, this is about 4KB, which is not a negligible size. The advantage on routing table size also holds for our algorithm.

The key difference between the elevator-first algorithm and ours is that the former allows any topology for the planar networks, while our algorithm narrows them to meshes. By restricting the topology, our algorithm eliminates virtual channel usage of the elevator-first algorithm which can lead to a significant reduction in energy consumption, or performance gain when the same number of virtual channels is used.

#### 4. PROPOSED ALGORITHM

In this section we show that, by composing some rules for choosing elevators (i.e., vertical links), the elevator-first algorithm can be made deadlock free even without any VCs at all. We start with a regular and easy architecture, and then give a routing algorithm for more general and irregular architectures.

##### 4.1. Preliminary

Before going into the details of the algorithm, let us define some necessary terms, most of which are from Dally and Seitz [1987] with slight modifications.

*Definition 1.* An *interconnection network*  $I = G(V, E)$  is a strongly connected graph where  $V$  represents the set of router nodes and  $E$  the set of channels. Each vertex is given  $(x, y, z)$ -coordinates according to their physical locations.

*Definition 2.* A *routing algorithm* is an algorithm that implements a routing function  $R: E \times V \rightarrow E$  that maps the current channel  $e_c$  and destination node  $v_d$  to the next channel  $e_n$  so that  $R(e_c, v_d) = e_n$ .

*Definition 3.*  $\text{src}(e_x)$  and  $\text{dest}(e_x)$  represent, respectively, the source node and destination node of  $e_x$ .

*Definition 4.* A *waiting path*  $P = \{e_0, e_1, \dots, e_k\}$  for routing function  $R$  is a sequence of channels where  $R(e_i, v_x) = e_{i+1}$  for some  $v_x$ . Thus, a flit in  $e_i$  which is to be routed to  $e_{i+1}$  may have to wait until  $e_{i+1}$  has room for it. Such dependency exists for all consecutive pairs of edges in  $P$ .

*Definition 5.* A *waiting cycle*  $C = \{e_0, e_1, \dots, e_k\}$  is a waiting path where  $e_k = e_0$ . There should be no waiting cycle in a deadlock-free routing.

<sup>1</sup>We also assume that the routing within the 2D plane can be implemented with combinational logic (e.g., XY routing).

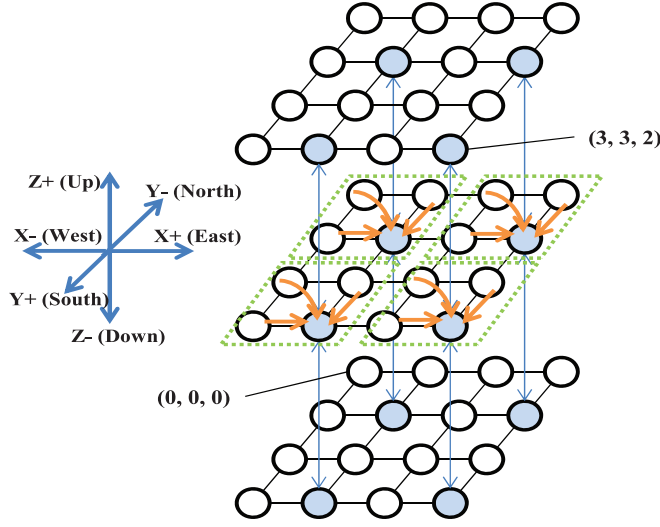


Fig. 1. An example of 3D stacked mesh topology with regular partial connections in the vertical direction.

**Definition 6.** A channel  $e_x$  is called an  $X+$  channel with respect to  $\text{src}(e_x)$  if  $\text{dest}(e_x)$  has a greater  $x$ -coordinate than  $\text{src}(e_x)$ .  $X-/Y+/Y-/Z+/Z-$  channels are also defined in the same way as depicted in Figure 1.  $X$  channels denote both  $X+/X-$  channels and, by the same token,  $Y$  and  $Z$  channels denote  $Y+/Y-$  and  $Z+/Z-$  channels, respectively. We also call the directions interchangeably as *east* ( $E$ ), *west* ( $W$ ), *north* ( $N$ ), *south* ( $S$ ), *up* ( $U$ ), and *down* ( $D$ ).

**Definition 7.** A turn  $T = (e_i, e_{i+1})$  consists of two differently directed channels  $e_i$  and  $e_{i+1}$  such that  $\text{dest}(e_i)$  equals  $\text{src}(e_{i+1})$ . The *turning node* is the common node of the two channels. “A path turns at node  $v$ ” means that the path has  $v$  as a turning node.

**Definition 8.** An  $\alpha\beta$  turn ( $\alpha, \beta: E, W, N, S, U, \text{ or } D$ ) represents a turn from  $\alpha$  direction to  $\beta$  direction. For example, an  $NE$  turn represents a turn from north direction to east direction.

**Definition 9.** A *row* is a set of nodes that share the same  $y$ - and  $z$ -coordinates. A *plane* or *layer* is a set of nodes that share the same  $z$ -coordinate.

**Definition 10.** A *primary direction* is one of the four planar directions ( $N, E, W$ , or  $S$ ) that is used to order the nodes within the routing algorithm. Once the primary direction is selected (e.g.,  $S$ ), then the secondary direction is one of the two planar directions that are not opposite to the primary direction (e.g.,  $E$  or  $W$ ). It is used to break ties when ordering nodes with the primary direction. Without loss of generality, we will use south ( $S$ ) as the primary direction and east ( $E$ ) as the secondary direction for the rest of this article.

**Definition 11.** A node  $v_x$  is called a *due-east* node of some other node  $v_y$  when  $v_x$  and  $v_y$  are on the same row and  $v_x$  is to the east of  $v_y$ . A node  $v_x$  is called a *south-or-due-east* node of some other node  $v_y$  when  $v_x$  is either to the south or to the *due-east* of  $v_y$ . A *north-or-due-west* node is defined similarly. The terms are also defined similarly for elevators, turns, and channels.

**Definition 12.** An *up-pivot* elevator of a plane is the up elevator that has no other up elevator to the primary direction of it in the plane. If there are multiple elevators

that satisfy the condition, the elevator farthest to the secondary direction is the pivot elevator. For example, if south is the primary direction and east in the secondary, then the up-pivot elevator has no other up elevator to its south-or-due-east. The down-pivot elevator is defined in the same way.

Note that an interconnection network  $I$  using routing function  $R$  is deadlock free if no waiting path in  $I$  forms a cycle. This has been proven in Dally and Seitz [1987] and we omit the proof here.

#### 4.2. Routing Algorithm for 3-D Stacked Meshes with Regular Partial Vertical Connections

Figure 1 shows our first architecture to start with. In Figure 1, each layer is a  $4 \times 4$  mesh and one vertical link is regularly placed at the south-east corner of every  $2 \times 2$  block. All vertical links in the figure are bi-directional (this is assumed throughout the article). In the architecture, deadlock freedom of the elevator-first algorithm can be obtained by one simple rule.

##### RULESET A

*Rule A1.* When the destination is on a different layer, take the elevator in the  $L \times M$  block ( $2 \times 2$  block in the example of Figure 1) that contains the source node to send the packet all the way to the destination layer.

The thick orange arrows in Figure 1 illustrate this rule. All packets from any of the four nodes within a  $2 \times 2$  block follow the XY route to the elevator in the same block. Although  $2 \times 2$  blocks are used in this example, the size of a block does not actually matter and any other sized block can be used, such as  $3 \times 3$  or  $2 \times 1$ , as long as each block has an elevator at its south-east corner. Now, we claim that Rule A1 makes the routing deadlock free, and the proof follows.

**LEMMA 1.** *In a routing algorithm following Rule A1, west-up (WU), west-down (WD), north-up (NU), and north-down (ND) turns never occur.*

**PROOF.** According to Rule A1, the  $x$ - and  $y$ -coordinates of an elevator to be taken are always equal to or greater than that of the source. If both  $x$ - and  $y$ -coordinates are equal, then a  $Z$  channel of the elevator is directly taken and no turn is made. If any of the coordinates are different, by the definition of XY routing, the  $E$  channel and  $S$  channel are the only channels that can be taken before the  $U/D$  channel. Thus, WU, WD, NU, and ND turns never occur.  $\square$

**THEOREM 1.** *An elevator-first routing algorithm with Ruleset A is deadlock free with no VC.*

**PROOF.** Assume a waiting path  $P = \{e_0, e_1, \dots, e_n\}$  following the elevator-first algorithm with Ruleset A.  $P$  is categorized into the following three cases.

*Case 1.*  $P$  does not include any  $Z$  channel. It is guaranteed by the property of XY routing that  $P$  cannot form a cycle.

*Case 2.*  $P$  includes one or more  $Z$  channels, but there is no  $Z$ -directed turn (a turn into a  $Z$  direction). In this case,  $P$  starts with a  $Z$  channel but never returns to the starting point and thus cannot form a cycle.

*Case 3.*  $P$  includes one or more  $Z$ -directed turns ( $EU$ ,  $ED$ ,  $SU$ , or  $SD$ ; by Lemma 1, a turn can never be a  $WU$ ,  $WD$ ,  $NU$ , or  $ND$  turn). Let  $T_1$  be such a  $Z$ -directed turn that is located on the *north-or-due-west* side of all other  $Z$ -directed turns in  $P$ . In other words, no other  $Z$ -directed turn in  $P$  is on the *north-or-due-west* side of  $T_1$ . Without loss of generality, let us assume that  $T_1$  is an *upward* turn; that is,  $T_1$  is either an  $EU$  turn or an  $SU$  turn. The rest of this proof shows it is impossible for  $P$  to form a cycle.

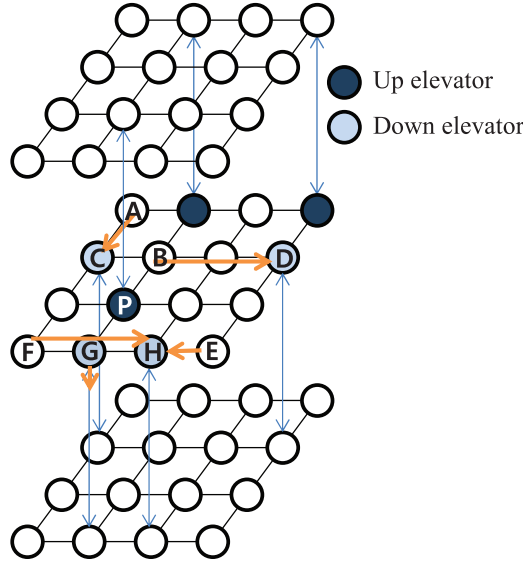


Fig. 2. 3D stacked meshes with random partial connections and the down elevator selection.

- (a) If  $T_1$  is an  $EU$  turn, there should be an  $E$ -directed turn just before  $T_1$  (consider the example of cycle  $UW$ - $WD$ - $DE$ - $EU$ ) in  $P$  to make a cycle, and the only turns allowed into  $E$  direction in  $XY$  routing are  $UE$  turn and  $DE$  turn. Moreover, to make a cycle, there should be a  $Z$ -directed turn in  $P$  which is followed by the  $UE$  or  $DE$  turn. However, since there is no other  $Z$ -directed turn on the *due-west* side of  $T_1$  by the assumption, no cycle can be formed.
- (b) If  $T_1$  is an  $SU$  turn, some turn  $T_3$  has to be made into  $S$  direction in  $P$  to make a cycle. It can be one of  $US$ ,  $DS$ ,  $ES$ , or  $WS$ . Obviously,  $US$  and  $DS$  turns are impossible because there is no other  $Z$ -directed turn on the north side of  $T_1$ .  $ES$  and  $WS$  turns are also impossible because only turns that can be made to  $E$  or  $W$  direction are from  $U$  or  $D$  directions, which would again violate the assumption that there is no other  $Z$ -directed turn on the north side of  $T_1$ . Thus no cycle can be formed and this proves the theorem.  $\square$

#### 4.3. Routing Algorithm for 3D Stacked Meshes with Irregular Partial Vertical Connections

One observation from Section 4.2 is that a block need not necessarily have the size of  $2 \times 2$ . Furthermore, a path does not need to take the vertical link in the same block. The only conditions used in the proof are: (1)  $XY$  routing is used for planar routing and (2) four turns ( $WU$ ,  $WD$ ,  $NU$ , and  $ND$ ) are prohibited at vertical links. Thus, provided that the two conditions are satisfied, we can easily extend the algorithm to the case of irregular placement of vertical links or to the case that allows adaptiveness of selectively taking elevators according to the condition of the network. We leave the issue of utilizing the adaptiveness as future work and focus in this section on more irregular architectures, where vertical links are randomly placed as shown in Figure 2.

Even if we have an irregular vertical link placement, we can make the routing algorithm take any elevator, provided that the aforementioned four turns are prohibited. However, this rule poses an important restriction on the architecture; a vertical link is required at the south-east corner node of the mesh. If the node does not have a vertical link, then some nodes will have no elevator provided by the routing algorithm. In other words, the routing algorithm results in a disconnected network. Fortunately, we can



solve the problem by additional rules that allow forbidden turns at certain nodes. With the addition of new rules and some adjustment to the existing rule, we can define a new ruleset for selecting an elevator.

### RULESET B

*Rule B1.* When the destination is on a different layer, take any elevator among the *south-or-due-east* ones of the source (including the self-node).

*Rule B2.* If there is no south-or-due-east elevator of the source in the desired direction, take the pivot elevator (refer to Definition 12 for the definition of pivot elevator).

*Rule B3.* Consider a case where a down elevator is chosen by Rule B1. If the down elevator is not at the self-node but located to the south-or-due-east of the up-pivot elevator, then instead of taking this down elevator, take the down-pivot elevator on the same plane. The same rule applies when up and down direction are switched.

Note that, in this ruleset, south represents the primary direction and east the secondary. They can be changed to other directions according to Definition 10. The thick arrows in Figure 2 show an example of elevator selection. According to Rule B1, packets from nodes A and B can choose down elevators C and D, respectively. However, packets from node E do not have any down elevator corresponding to Rule B1, and thus they choose the down-pivot elevator H following Rule B2. The addition of Rule B3 eliminates the deadlock problem caused by adding Rule B2. Normally, packets having node F as their source would take node G as their down elevator by Rule B1. However, because G is located to the south of P (the up-pivot elevator), the down-pivot elevator at node H is selected instead, according to Rule B3. On the other hand, packets having G as their source take the down elevator at the self-node, G. We claim this routing algorithm is deadlock free. Please note that the pivot elevator is one of the existing elevators, not a newly added one.

**LEMMA 2.** *In a waiting path  $P = \{e_0, e_1, \dots, e_n\}$  constructed following Ruleset B, let  $e_i$  ( $0 \leq i \leq n$ ) be an up segment and  $e_j$  ( $0 \leq j \leq n$ ) be a down segment in  $P$ , where  $i < j$  and  $\text{src}(e_i)$  and  $\text{dest}(e_j)$  are on the same plane. Then  $\text{dest}(e_j)$  is to the south-or-due-east of  $\text{src}(e_i)$ . The same holds when  $e_i$  is a down segment and  $e_j$  an up segment.*

**PROOF.** We prove by induction. Without loss of generality, let  $e_i$  be an up segment. We will first show that Lemma 2 is true when there is only one plane above  $\text{src}(e_i)$ , and then show that if it is true when there are  $n$  planes above it, it is also true when there are  $n+1$  planes above it. The illustration of the proof is shown in Figure 3. If there is only one plane above  $\text{src}(e_i)$ , by Rule B1, the down elevator that can be taken must be on the *south-or-due-east* side of  $\text{dest}(e_i)$  (the only exception is Rule B2, but an elevator on the *north-or-due-west* side of  $\text{dest}(e_i)$  cannot be a pivot elevator and thus cannot be taken). Therefore,  $e_j$  (and thus  $\text{dest}(e_j)$ ) must be on the *south-or-due-east* side of  $e_i$  (and thus  $\text{src}(e_i)$ ). Now, let's assume that the lemma is true when there are  $n$  planes upward. Assume first that the path between  $e_i$  and  $e_j$  never reaches the plane above that of  $\text{dest}(e_i)$ . Then  $e_j$  is to the south or due east of  $e_i$  just as the case of only one upper plane existing. Now, assume that the path has an upward channel  $e_k$  ( $i < k < j$ ) after  $e_i$  and  $\text{dest}(e_i)$  and  $\text{src}(e_k)$  are on the same plane. Then there can be two cases.

*Case 1.*  $e_k$  is on the *south-or-due-east* side of  $e_i$  or at the same  $(x, y)$  location as  $e_i$ . Assume that there are  $n$  planes above  $\text{src}(e_k)$ , then the down channel  $e_l$  ( $k < l < j$ ) in  $P$  has destination node on the same plane as  $\text{src}(e_k)$ , and  $e_l$  is on the *south-or-due-east* side of  $e_k$ . Then  $e_j$  must be on the *south-or-due-east* side of  $e_i$ . Otherwise,  $e_j$  should have been a down-pivot elevator since it should have been taken by Rule B2 after  $e_l$  even if it were located on the *west-or-due-north* side of  $e_l$  (recall the relative locations of  $e_i$  and

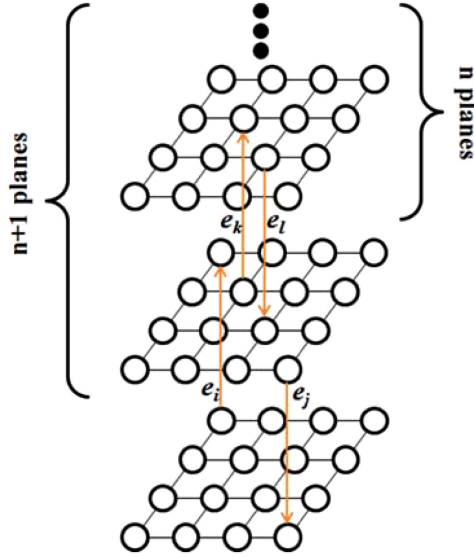


Fig. 3. Illustration of the Proof of Lemma 2.

$e_l$ ). In this case, there would have been violation of Rule B3 since  $e_k$  is not an up-pivot elevator (the up elevator at the location of  $e_l$  is actually the up-pivot elevator).

*Case 2.*  $e_k$  is a *north-or-due-west* channel of  $e_i$ . This means  $e_k$  is the up-pivot elevator. Assume the down channel  $e_l$  ( $k < l < j$ ) in  $P$  has destination node on the same plane as  $\text{src}(e_k)$ , and that  $e_l$  is a *south-or-due-east* channel of  $e_k$ . This is contradiction to the fact that  $e_k$  is an up-pivot elevator (up elevator at the location of  $e_l$  is the up-pivot elevator). Thus this case never occurs.

Therefore, by induction we can say that the lemma always holds.  $\square$

**THEOREM 2.** *The elevator-first routing algorithm with Ruleset B is deadlock free with no VC.*

**PROOF.** Let  $P = \{e_0, e_1, \dots, e_n\}$  be a waiting path constructed following Ruleset B. If there is no Z channel in  $P$ , then no cycle can be formed by the property of XY routing. Now assume that there are one or more Z channels in  $P$  and let the first Z-directed turn in  $P$  be  $T_1 = (e_i, e_{i+1})$  and its turning node be  $t_1$ . By Lemma 2, any other Z channel in  $P$  that has destination node on the same plane as  $t_1$  is on the *south-or-due-east* side of  $t_1$ . If  $T_1$  is one of *NU*, *ND*, *WU*, or *WD* turns, this means that  $e_{i+1}$  is the pivot elevator. However, by Lemma 2 again, any Z segment in  $P$  having destination on the same plane as  $t_1$  must be on the *south-or-due-east* side of  $t_1$  (or  $e_{i+1}$ ), implying that  $e_{i+1}$  cannot be a pivot elevator, which is a contradiction. Thus this case never happens. If  $T_1$  is one of *EU*, *ED*, *SU*, or *SD* turns,  $T_1$  cannot be reached from *south-or-due-east* nodes by the XY routing algorithm. Therefore  $P$  cannot form a cycle, and the algorithm is deadlock free.  $\square$

#### 4.4. Extension to Heterogeneous Mesh Layers

In 3D stacking technology, each layer can be used for different purposes and thus different technology can be used. For example, processor cores can be placed on one layer and memory on another [Akesson et al. 2011]. Consequently, maintaining the same network topology in each layer can be difficult in some situations. In this section,

we show that the same routing algorithm can be used even for heterogeneously sized mesh layers while maintaining deadlock freedom if the topology preserves the “relative locations” between elevators. Informally, preservation of relative locations can be explained as follows. An elevator is said to connect a source router and a destination router at source layer and destination layer, respectively. In a topology where the relative locations of elevators are preserved, if the source routers of two elevators have the same  $x$ -coordinate, the destination routers of the corresponding elevators should also have the same  $x$ -coordinate. Similarly, if the source router of an elevator has a larger  $x$ -coordinate than the other source router, the destination router of this elevator should also have a larger  $x$ -coordinate. Of course, the same holds for the  $y$ -coordinates. Under this condition, all characteristics that we used to prove Ruleset B hold for heterogeneously sized meshes. Considering that tilted TSVs are usually not implemented for any reason, preserving the relative locations does not pose a restriction to the topology. To make a formal definition, we first define two sets  $SX_n$  and  $SY_n$  as follows.

- $SX_n = \{sx_{ni} \mid v_i \text{ is a node in layer } n, v_i \text{ is connected by a vertical link, and } sx_{ni} \text{ is the } x\text{-coordinate of } v_i\}$ .
- $SY_n = \{sy_{ni} \mid v_i \text{ is a node in layer } n, v_i \text{ is connected by a vertical link, and } sy_{ni} \text{ is the } y\text{-coordinate of } v_i\}$ .

We also define two mappings  $f$  and  $g$  as follows.

- $f: SX_n \rightarrow SX_m$  is a mapping between  $x$ -coordinates in layer  $n$  and those in layer  $m$  implemented by vertical links. Thus,  $sx_{mj} = f(sx_{ni})$  represents a vertical link from a node with  $x$ -coordinate  $sx_{ni}$  to a node with  $x$ -coordinate  $sx_{mj}$ .
- $g: SY_n \rightarrow SY_m$  is a mapping between  $y$ -coordinates in layer  $n$  and those in layer  $m$  implemented by vertical links.

Thus, if there is a vertical channel  $e_i = (v_i, v_{i+1})$  and  $v_i$  is located at  $(x, y)$ , then  $v_{i+1}$  is located at  $(f(x), g(y))$ . The mappings  $f: SX_n \rightarrow SX_m$  and  $g: SY_n \rightarrow SY_m$  must be monotone increasing functions for any layers  $n$  and  $m$ . That is, the mappings must satisfy the following topological conditions:

$$\begin{aligned}
 sx_{n1} > sx_{n2} &\Leftrightarrow f(sx_{n1}) > f(sx_{n2}), \\
 sx_{n1} = sx_{n2} &\Leftrightarrow f(sx_{n1}) = f(sx_{n2}), \\
 sx_{n1} < sx_{n2} &\Leftrightarrow f(sx_{n1}) < f(sx_{n2}), \\
 sy_{n1} > sy_{n2} &\Leftrightarrow g(sy_{n1}) > g(sy_{n2}), \\
 sy_{n1} = sy_{n2} &\Leftrightarrow g(sy_{n1}) = g(sy_{n2}), \text{ and} \\
 sy_{n1} < sy_{n2} &\Leftrightarrow g(sy_{n1}) < g(sy_{n2}).
 \end{aligned}$$

Figure 4 shows two examples: Figure 4(a) satisfies the topological conditions while (b) does not. This set of restrictions maintains relative locations of nodes connected by vertical links in a 3D structure of meshes of different size and shape. Because all TSVs are actually manufactured perpendicular to the dies, the conditions are practically not restrictions.

**THEOREM 3.** *The routing algorithm described in Section 4.3 on heterogeneously sized mesh architecture that satisfies the topological conditions is deadlock free with no VC.*

**PROOF.** The proof is done by renumbering the coordinates. Let us choose the topmost layer  $l$  as a reference layer. For the adjacent layer  $m$ , there exist functions  $f$  and  $g$  such that  $sx_{mj} = f(sx_{li})$  and  $sy_{mj} = f(sy_{li})$ . Then, renumber all coordinates of the connected nodes in  $m$  and  $l$  such that  $sx_{mj} = sx_{li}$  and  $sy_{mj} = sy_{li}$ . The renumbering is done by assigning the bigger value to each pair (e.g., if  $sx_{mj}$  is bigger, then the value is assigned to  $sx_{li}$ ). Give all other nodes proper coordinate values relative to the nodes already

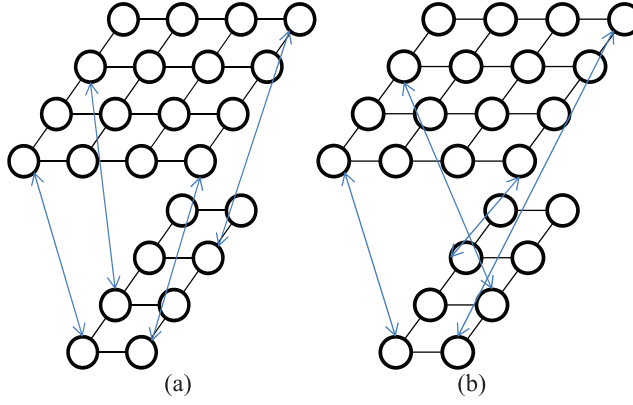


Fig. 4. Examples of heterogeneous mesh architecture: (a) An architecture that satisfies the topological conditions; (b) the other one that does not.

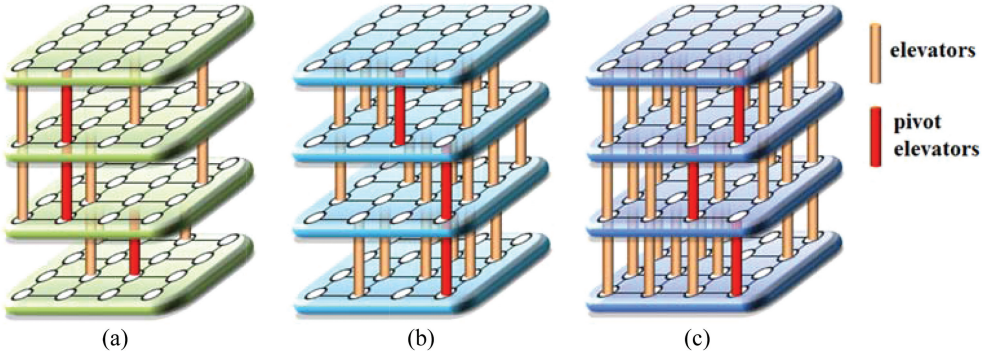


Fig. 5. Partially connected mesh architectures for experiments: (a), (b), and (c) have 25%, 50%, and 75% of vertical connections.

renumbered (this process is similar to image warping). In this way, all the layers can be renumbered. Then, all the Z-directed channels have their source and destination nodes with matching  $(x, y)$ -coordinates. Then all the architectural assumptions used for Theorem 2 still hold, and thus Theorem 3 can be proven in the same way.  $\square$

## 5. EVALUATION

### 5.1. Experimental Setup

We performed experiments using 3D NoC architectures consisting of four layers of  $4 \times 4$  mesh topology as shown in Figure 5. The core clock frequency and NoC clock frequency are, respectively, assumed to be 2.6 GHz and 2.0 GHz. The router was 3-stage pipelined. Each port has 8-flit buffers and the buffer bypass technique was used. To evaluate 3D NoCs under various conditions of partial vertical connections, we randomly generated three partial connected 3D mesh architectures which have 25%, 50%, and 75% of vertical connections. To examine NoC performance, an in-house, cycle-accurate NoC simulator was used. For energy comparison, DSENT [Sun et al. 2012], an NoC power modeling tool, was used to model the components of NoC routers and wires. For TSVs, the model from Ouyang et al. [2010] was used. Table I shows the parameters used for the energy model in this article.

For the benchmarks, synthetic traffic patterns as well as traces from real applications were used. Four kinds of synthetic traffic pattern were used: uniform random, hotspot

Table I. Network Design Parameters

| Technology node             | 45nm                  |
|-----------------------------|-----------------------|
| Network size                | $4 \times 4 \times 4$ |
| Frequency                   | 2.0GHz                |
| TSV length                  | $100\mu\text{ m}$     |
| Wire length                 | 1mm                   |
| Router pipeline latency     | 2 cycles              |
| Wire latency                | 1 cycles              |
| Buffers per virtual channel | 8 flits               |
| Channel width               | 128 bits              |

random, bit-complementary, and tornado. Under these kinds of traffic, three metrics, including average latency, aggregate throughput, and energy consumption per flit, were measured. In the simulation, the networks were warmed up for 100,000 cycles and run for another 100,000 cycles. After that, the networks were run without injecting any more packets into the network until all packets reached the destination to ensure there was no deadlock. For real applications, traces of 21 multithreaded applications from SPLASH-2 [Woo et al. 1995] and PARSEC [Bienia et al. 2008] were used. For the evaluation of real applications, we assumed each router is connected to a node that has an x86 processor, a private L1 cache, and a slice of shared L2 cache. The L2 cache is configured as a low-bit interleaved S-NUCA [Kim et al. 2002] and assumed perfect. Traffic from cache misses as well as coherence messages were modeled and used to stress the 3D NoC network.

For the evaluation of the proposed algorithm, we consider the three routing algorithms with corresponding NoC architecture as follows.

- Elf*. This baseline elevator-first algorithm was proposed in Dubois et al. [2013]. It uses two VCs for the planar links.
- Redelf*. The proposed algorithm in this article with no VCs was used.
- Redelfv2*. This is the proposed algorithm using two planar VCs. Since the baseline elevator-first algorithm (i.e., *Elf*) uses two VCs for the planar links, comparing it with the proposed algorithm (i.e., *Redelf*) wouldn't be fair in the view of performance. Obviously, the network with more VCs would be better in latency and throughput while suffering the power and area overhead of larger buffers and VC allocators. Thus, we also tested the proposed algorithm in this article with two planar VCs in order to compare it with the baseline elevator-first algorithm. However, instead of using the VCs to avoid deadlock, we used them to mitigate head-of-line blocking.

Unlike synthetic traffic patterns, real application traffic requires more VCs to avoid protocol deadlock in addition to the ones needed for routing deadlock avoidance. The coherence protocol we use requires two VCs, one for request messages and the other for reply messages. Thus, for the real application benchmarks, usage of virtual channels is doubled, which gives two for *Redelf* and four for *Elf* and *Redelfv2*.

For real application workloads, we compare our algorithm with an additional routing algorithm, *Mroots*, designed for arbitrary topology.

- Mroots*. The problem of up\*/down\* routing is its traffic congestion around the root. *Mroots* uses virtual channels given for protocol deadlock avoidance to distribute traffic across multiple roots. Within each virtual channel, packets are routed using up\*/down\* routing with different spanning trees [Lysne et al. 2006]. The roots of the trees are placed as far as possible to avoid overlay of congested regions. In our experiment, each of the message types (i.e., request and reply) required for the

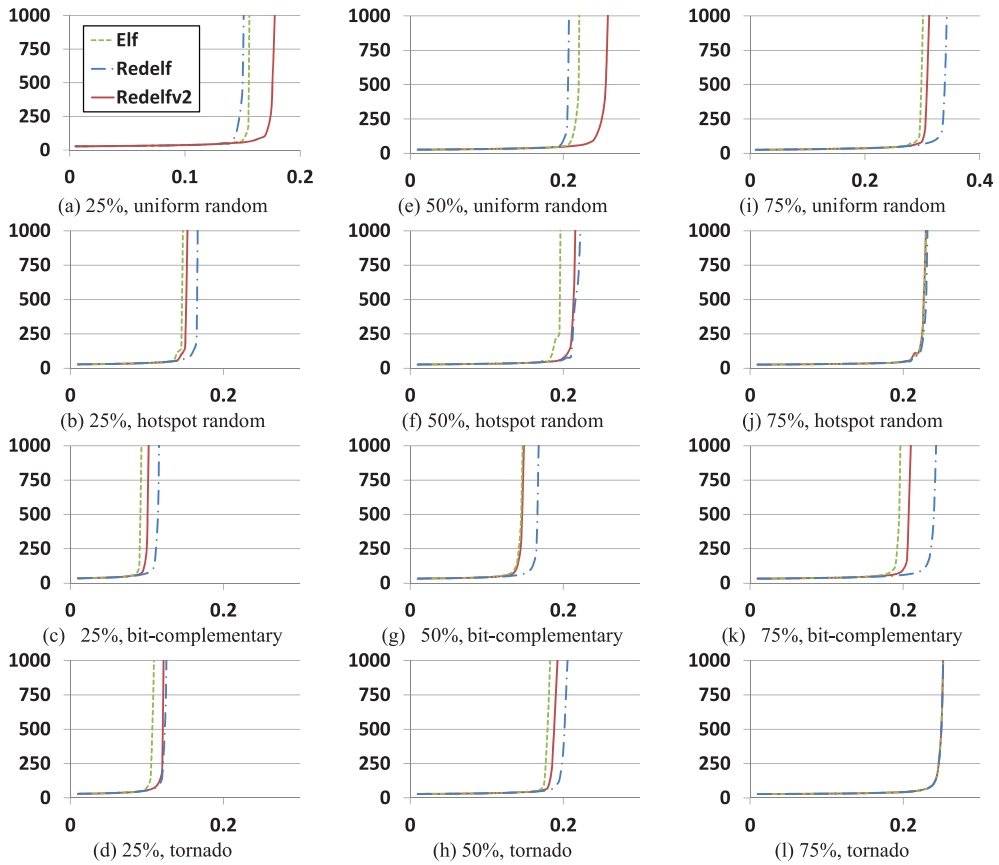


Fig. 6. Comparison on average latencies over various number of vertical connections and traffic patterns. X axis represents injection ratio (flits/node/cycle) and Y axis represents average latency (cycles).

coherence protocol has its own dedicated independent virtual channel. Therefore Mroots uses two virtual channels in total, which is the same as Redelf.

## 5.2. Experiments on Synthetic Traffic

Figure 6 shows latency results of the routing algorithms (i.e., Elf, Redelf, and Redelfv2) under different traffic patterns (i.e., uniform random, hotspot random, bit-complementary, and tornado) on a 3D NoC with different number of vertical links (i.e., 25%, 50%, and 75%). As shown in Figure 6, Redelfv2 shows the best latency in all cases. This comes mainly from better utilization of the VCs. Even though Elf also uses two VCs, the VCs can be unbalanced because the direction of the packets decides which VC the packet would take. Redelf shows the longest latency, which certainly comes from the lack of VCs. However, the difference between Redelf and Elf is relatively small. To make a quantitative comparison, we can define *saturated point* as the point where latency reaches over 500 cycles. On average, the saturation point of Redelfv2 is 8.4% higher than Elf, and that of Redelf is 4.5% lower than Elf.

Figure 7 shows the aggregate throughput of each routing algorithm. In Figure 7, Redelfv2 clearly shows the best results. One important observation is that in many cases, Elf and Redelf show severe throughput drop after they reach top performance as the injection ratio increases, which comes from congestion at the vertical channels

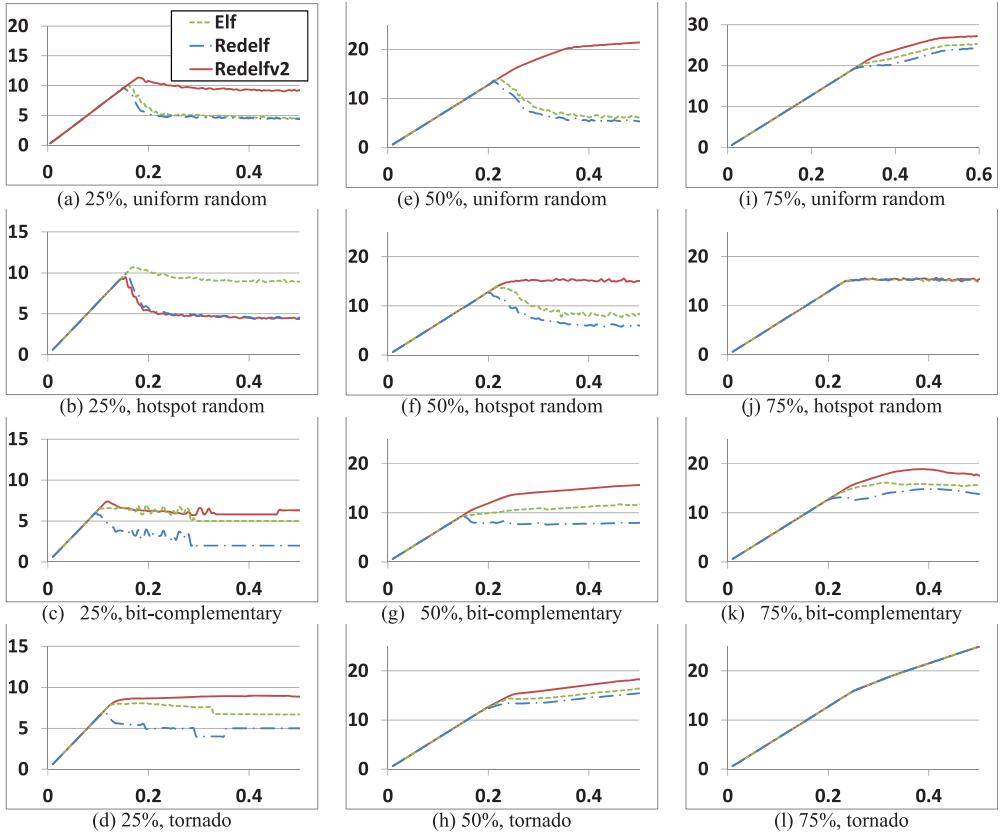


Fig. 7. Comparison on aggregate throughput over various number of vertical connections and traffic patterns. The  $x$ -axis represents injection ratio (flits/node/cycle) and  $y$ -axis represents aggregate throughput (flits/cycle).

when there are small number of vertical channels. This is rarely observed in Redelfv2 and the effect is not severe. Because overshooting is observed in the graphs, we compare the throughput in terms of peak and saturated throughput. In average peak throughput, Redelfv2 is 13.7% higher than Elf, and Redelf is only 6.0% lower than Elf. In terms of average saturated throughput, Redelfv2 is 31.2% higher than Elf, and Redelf is 10.6% lower than Elf. In summary, our algorithm is superior to the baseline algorithm when the same amount of resources (i.e., VCs) is used and, even when used without VC, the performance drop is relatively small.

Figure 8 shows the results of *energy consumption per flit* (EPF) for each routing algorithm. At low injection ratio, the EPF results are very high because the static energy of the network dominates total energy consumption. As the injection ratio increases, the EPF results drop because the dynamic energy dominates. Under those conditions where the throughput drops, the EPF results also drop. Mainly because of throughput differences, Redelfv2 tends to show lower EPF than Elf. Redelf usually consumes the lowest EPF, especially under low injection ratio. This comes from the smaller hardware overhead. However, under some conditions such as the one with uniform random traffic on a 50% vertical channel architecture, EPF starts to rise after some point because the throughput drops and so does energy efficiency. In terms of lowest EPF, Redelf consumes 27.9% less energy than Elf on average, and Redelfv2

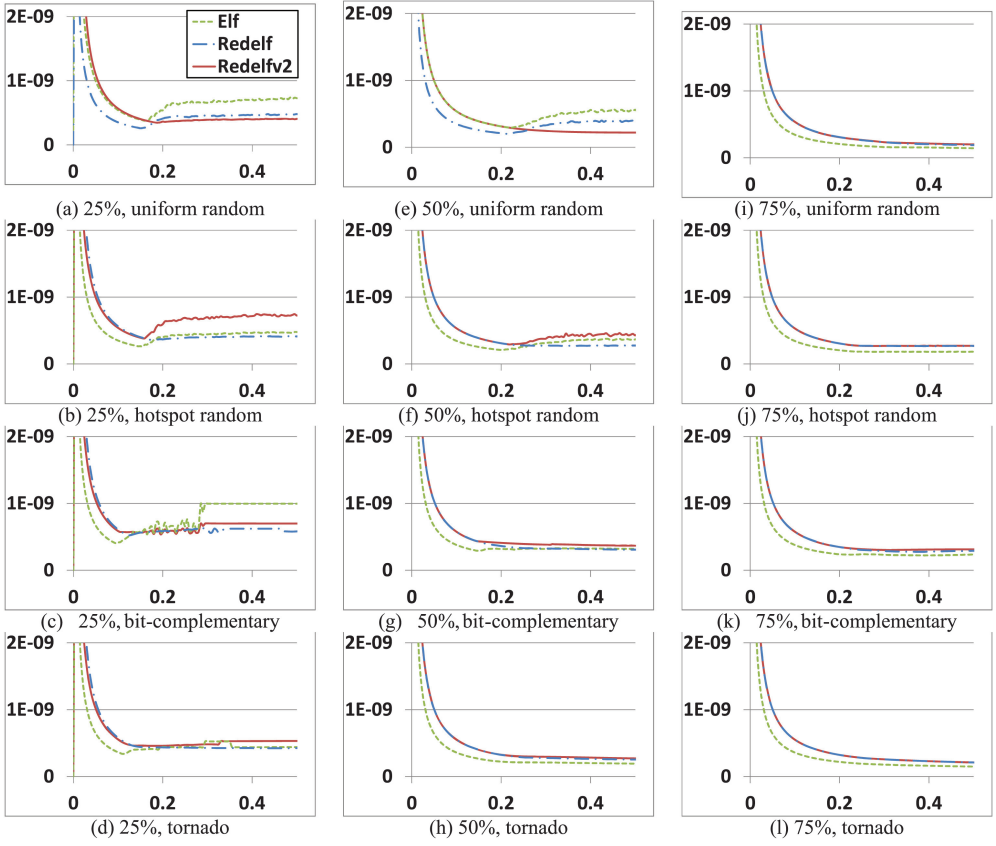


Fig. 8. Comparison of EPF over various number of vertical connections and traffic patterns. The  $x$ -axis represents injection ratio (flits/node/cycle) and  $y$ -axis represents EPF (J/flit).

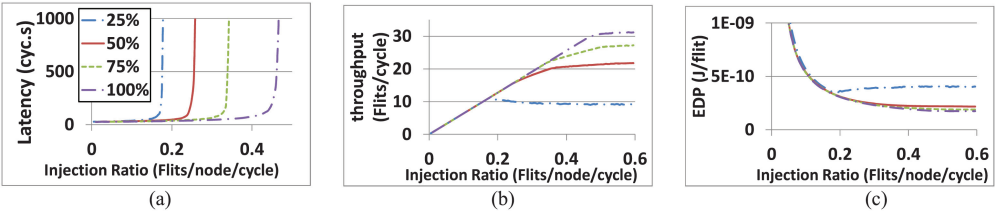


Fig. 9. Comparison of average latency, aggregate throughput, and EPF over various number of vertical connections under uniform random traffic pattern.

consumes 8.0% less than Elf. For saturated EPF, Redelf is 17.7% lower than Elf and Redelfv2 is 27.7% lower than Elf. This implies that, when designing for low power, Redelf has the advantage in terms of energy efficiency when the network is expected to work at relatively low load, whereas Redelfv2 should be better when there is high stress in the network.

Figure 9 shows how the reduced number of vertical links directly affects system performance of the network. Because the small number of vertical links limits the bi-section bandwidth, the throughput/latency is roughly proportional/inversely proportional to the number of vertical links. EPF, on the other hand, shows less difference depending on the number of vertical links. This is because the energy efficiency gained



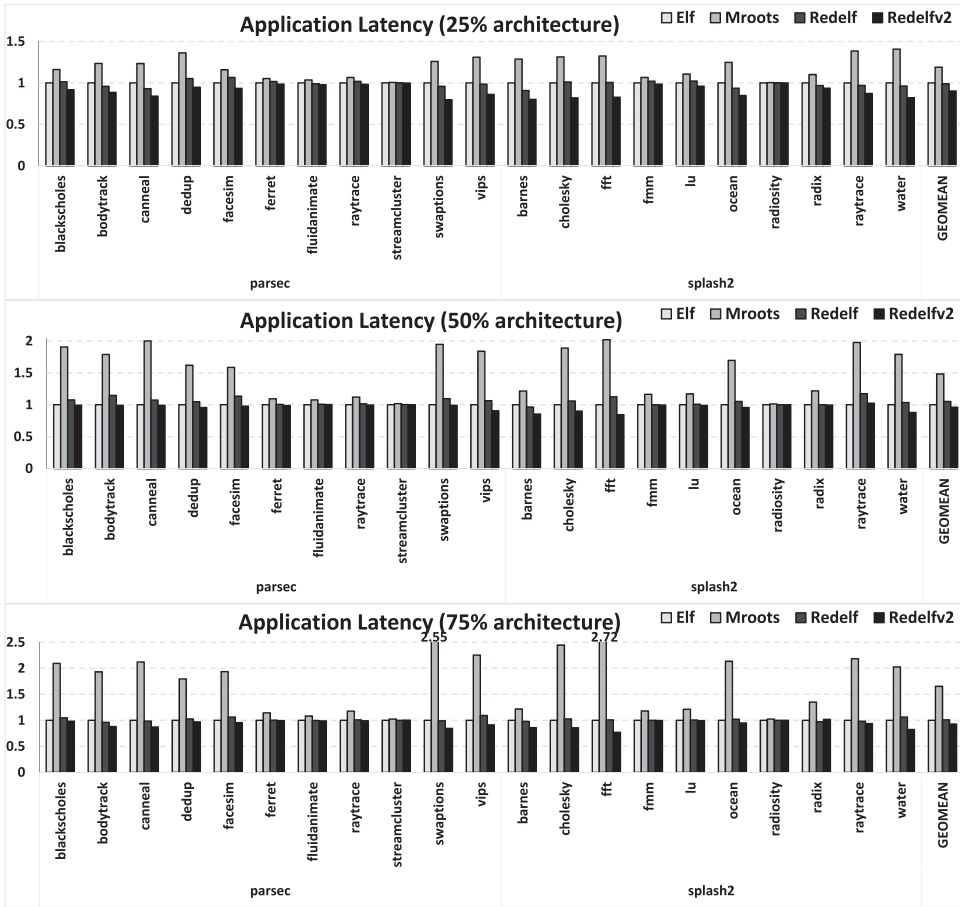


Fig. 10. Application latencies on different routing algorithms. The values are normalized to Elf in each graph.

from the increased throughput is offset by the increased power consumption due to more links, buffers, and logic components.

### 5.3. Experiments on Application Benchmarks

Figure 10 shows the application latency (execution time) results of the routing algorithms (i.e., Elf, Mroots, Redelf, and Redelfv2) on a 3D NoC with different number of vertical links (i.e., 25%, 50%, and 75%) for real applications. As expected, Redelfv2 shows the best latency in all cases, up to 23.1% (on average, 6.9%) reduction in latency compared to Elf, because of the better utilization of VCs. Although Redelfv2 uses the same number of VCs as Elf, it has more freedom to assign packets to VCs, while Elf has fixed assignment to avoid routing-induced deadlock as well as protocol-induced deadlock. Compared to Elf, Redelf shows only less than 10.0% (on average, 1.7%) increase in latency. Mroots shows the worst performance because its spanning-tree-based routing incurs too much traffic congestion around the two roots. Furthermore, their two virtual channels are allocated to the request and reply message channels, one for each, and they do not try to balance traffic among the virtual channels. Mroots shows 42.9% longer latency on average. Table II shows some hints on the latency results. It shows the average zero-load latencies of the routing algorithms under uniform random traffic, and

Table II. Zero-Load Latencies (cycles)

| Routing \ Architecture | 25%  | 50%  | 75%  |
|------------------------|------|------|------|
| Elf                    | 21.9 | 20.5 | 19.7 |
| Redelf                 | 22.4 | 21.1 | 20.1 |
| Mroots                 | 29.1 | 25.5 | 25.4 |
| Ideal                  | 19.6 | 18.2 | 17.6 |

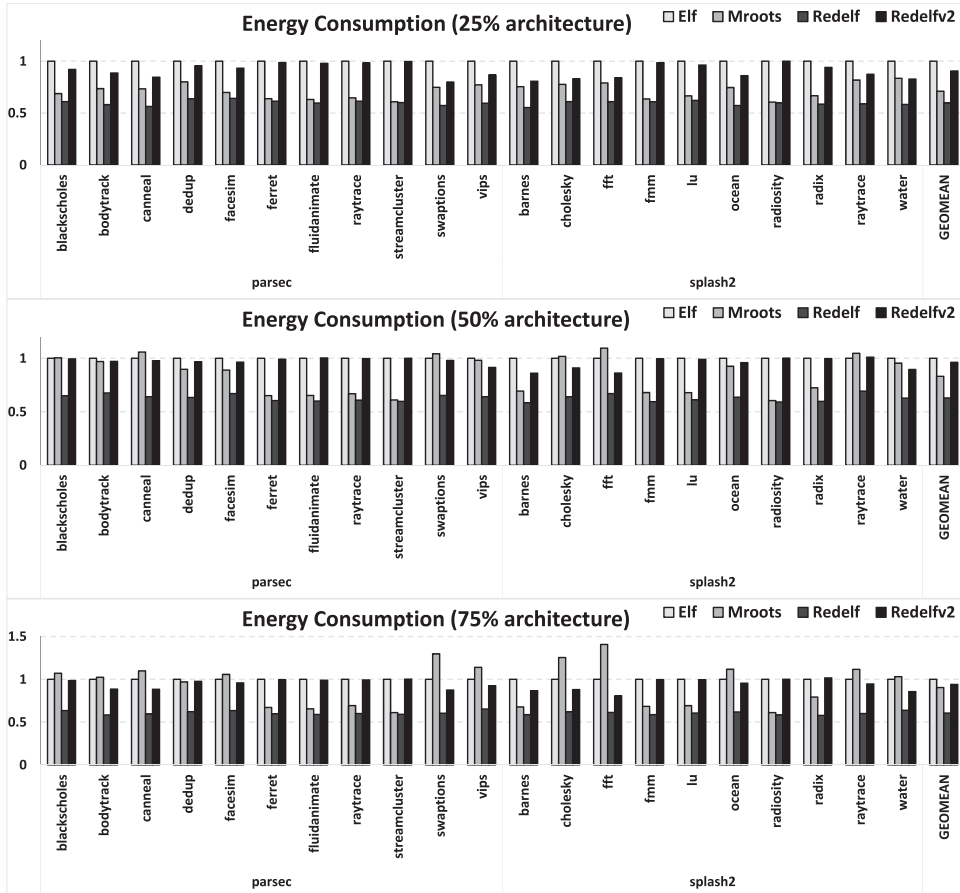


Fig. 11. Energy consumption on different routing algorithms. The values are normalized to Elf in each graph.

the ideal latency obtained by the shortest-path algorithm. Because Redelf has some restriction on its elevator selection, its zero-load latency is longer than that of Elf. This, together with the lack of virtual channel, makes Redelf slower than Elf. However, Redelfv2 beats Elf, despite the longer zero-load latency, because it has more balanced traffic across virtual channels. Mroots, however, shows the longest the application latency since it has long zero-load latency and still suffers near-root congestion even with the two roots. One more thing to note is that the difference between ideal zero-load latency and the latencies of the others is smaller on the 75% mesh topology, because the effect of restriction on selecting elevators becomes weak as the number of elevators increases.

Figure 11 shows the results for energy consumption. Redelf yields up to 44.6% (on average, 38.9%) reduction in energy consumption compared to Elf, mainly due to the

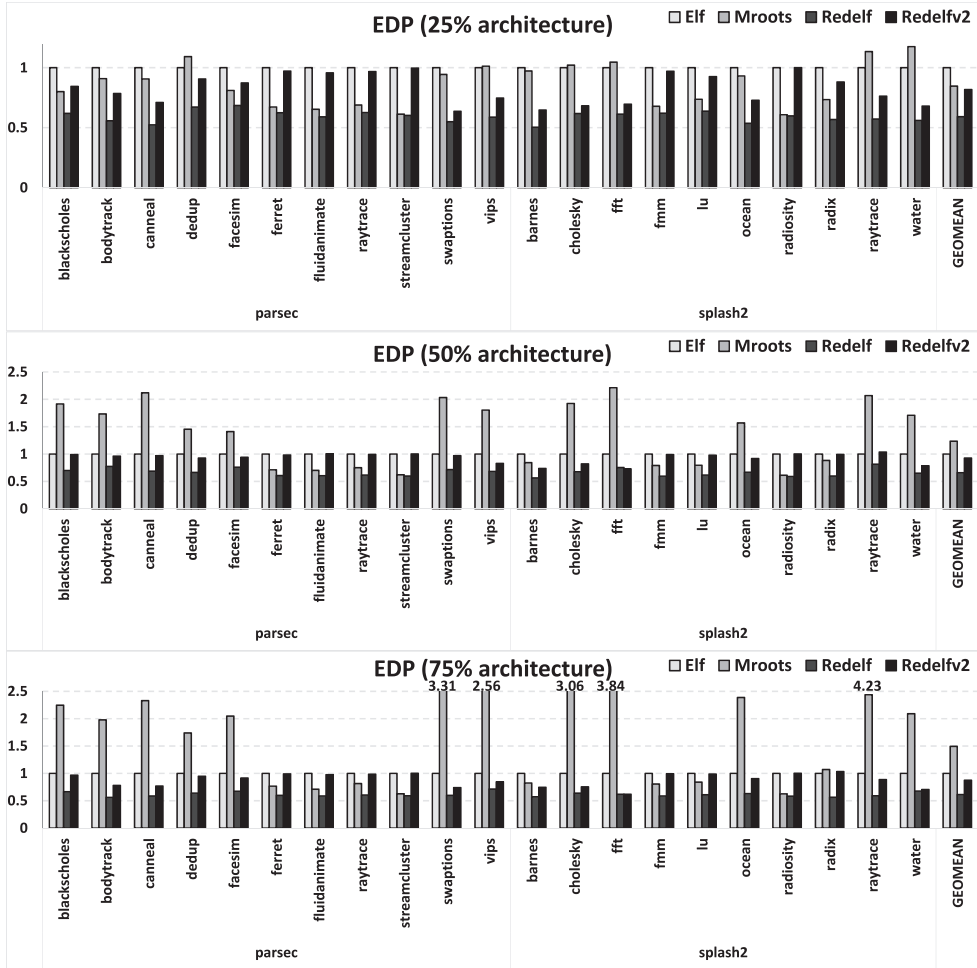


Fig. 12. Energy-delay product of different routing algorithms. The values are normalized to Elf in each graph.

reduction in VCs. The buffers in VCs take a very large portion of the energy consumption in NoC routers. Even though the running time increases, power reduction coming from the elimination of VC allocation logic, together with less number of buffers, does more than offset the excess running time, resulting in total energy reduction. Thus Redelf gains significant energy efficiency. Compared to Elf, Redelfv2 yields up to 20.1% (on average, 6.4%) reduction in energy consumption, even though it uses the same number of VCs as Elf. This is mainly due to the reduced running time. Mroots has low static power and dynamic energy per access because it uses a relatively small amount of hardware resources, like Redelf. However, the reduction due to small hardware is mostly offset by the long running time. As a result, the average energy savings of Mroots over Elf is only 18.8%, which is 20.1% less than that of Redelf.

Considering that there is always a trade-off between energy and latency, we use EDP (*energy-delay product*) as a metric to represent energy efficiency. The EDP over the baseline is shown in Figure 12. Redelf shows up to 49.6% EDP reduction and 37.9% reduction on average from the baseline Elf. The reduction mainly comes from the energy consumption of less virtual channels and buffers. The marginal performance drop does

not affect much of the EDP metric. Redelfv2, on the other hand, obtains reduction in both execution time and energy consumption. As a result, Redelfv2 shows up to 36.3% reduction and 12.8% on average compared to Elf. In the case of Mroots, large performance loss and small energy reduction result in 16.0% increase in EDP over the baseline Elf. Advantage in EDP does not always mean superiority because reducing latency is sometimes much harder than reducing energy or vice versa. However, the significant difference in EDP implies that our design uses energy far more efficiently.

It is also worth noting that the benefits from Redelf get slightly worse as the number of elevators increases. This is because, as the number of elevators increases, the 3D NoC becomes similar to ordinary meshes (i.e., 3D cube), and congestion at the elevators becomes less severe. As a result, the latency reduction becomes smaller, as well as the reduction in static energy consumption. However, the trend remains the same throughout all 3D NoC architectures.

## 6. CONCLUSIONS

In this article, we propose a deadlock-free routing algorithm for 3D NoCs with partial vertical connections, without using any VCs. Proofs are given and the experimental results show the benefits from the proposed routing algorithm in view of energy efficiency and performance. Compared to the conventional routing algorithm (i.e., baseline elevator-first algorithm), the proposed algorithm shows improvement of 38.9% in energy consumption, and 6.9% in performance.

Partially connected architectures are likely to be used because of the high cost of TSVs. However, as can be seen from the experimental results, their reduced vertical bandwidth can easily become a bottleneck. We believe this can be mitigated by mapping tasks/data into nodes efficiently or by using adaptive routing based on the proposed idea, which is our future work.

## REFERENCES

- Benny Akeson, Po-Chun Huang, Fabien Clermidy, Denis Dutoit, Kees Goossens, Yuan-Hao Chang, Tei-Wei Kuo, Pascal Vivet, and Drew Wingard. 2011. Memory controllers for high-performance and real-time MPSoCs: Requirements, architectures, and future trends. In *Proceedings of the International Conference on Hardware/Software Codesign and System Synthesis*. 3–12.
- Maryam Bahmani, Abbas Sheibanyrad, Frederic Petrot, Florentine Dubois, and Paolo Durante. 2012. A 3D-NoC router implementation exploiting vertically-partially-connected topologies. In *Proceedings of the IEEE Computer Society Annual Symposium on VLSI*. 9–14.
- Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: Characterization and architectural implications. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*. 72–81.
- Shekhar Borkar, Robert Cohn, George Cox, Sha Gleason, and Thomas Gross. 1988. Warp: An integrated solution of high-speed parallel computing. In *Proceedings of the International Conference on Supercomputing*. 330–339.
- Nizar Dahir, Ra'ed Al-Dujaily, Alex Yakovlev, Petros Missailidis, and Terrence Mak. 2012. Deadlock-free and plane-balanced adaptive routing for 3D networks-on-chip. In *Proceedings of the International Workshop on Network-on-Chip Architectures*. 31–36.
- William J. Dally and Charles L. Seitz. 1987. Deadlock-free message routing in multiprocessor interconnection networks. *IEEE Trans. Comput.* 100, 547–553.
- Florian Darve, Abbas Sheibanyrad, Pascal Vivet, and Frederic Petrot. 2011. Physical implementation of an asynchronous 3D-NoC router using serial vertical links. In *Proceedings of the IEEE Computer Society Annual Symposium on VLSI*. 25–30.
- Florentine Dubois, Abbas Sheibanyrad, Frederic Petrot, and Maryam Bahmani. 2013. Elevator-first: A deadlock-free distributed routing algorithm for vertically partially connected 3D-NoCs. *IEEE Trans. Comput.* 62, 609–615.
- Chris Fallin, Greg Nazario, Xiangyao Yu, Kevin Chang, Rachata Ausavarungnirun, and Onur Mutlu. 2012. MinBD: Minimally-buffered deflection routing for energy-efficient interconnect. In *Proceedings of the International Symposium on Networks-on-Chip*. 1–10.

- Jose Flich, Tor Skeie, Andres Mejia, Olav Lysne, Pedro Lopez, Antonio Robles, Jose Duato, Michihiro Koibuchi, Tomas Rokicki, and Jose Carlos Sancho. 2012. A survey and evaluation of topology-agnostic deterministic routing algorithms. *IEEE Trans. Parallel Distrib. Syst.* 23, 405–425.
- Sahar Foroutan, Abbas Sheibanyrad, and Frederic Petrot. 2014. Assignment of vertical-links to routers in vertically-partially-connected 3-D-NoCs. *IEEE Trans. Comput.-Aid. Des. Integr. Circ. Syst.* 33, 1208–1218.
- Yatin Hoskote, Sriram Vangal, Arvind Singh, Nitin Borkar, and Shekhar Borkar. 2007. A 5-GHz mesh interconnect for a teraflops processor. *IEEE Micro* 27, 51–61.
- Andrew B. Kahng, Bin Li, Li-Shiuan Peh, and Kambiz Samadi. 2009. ORION 2.0: A fast and accurate NoC power and area model for early-stage design space exploration. In *Proceedings of the Design, Automation, and Test in Europe Conference (DATE'09)*. 423–428.
- Kyungsu Kang, Luca Benini, and Giovanni De Micheli. 2014. Cost-effective design of mesh-of-tree interconnect for multi-core clusters with 3-D stacked L2 scratchpad memory. *IEEE Trans. VLSI Syst.* (to appear).
- Nishit Kapadia and Sudeep Pasricha. 2012. A power delivery network aware framework for synthesis of 3D networks-on-chip with multiple voltage islands. In *Proceedings of the International Conference on VLSI Design (VLSID'12)*. 262–267.
- Changkyu Kim, Doug Burger, and Stephen W. Keckler. 2002. An adaptive, non-uniform cache structure for wire-delay dominated on-chip caches. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'02)*. 211–222.
- Jongman Kim, Chrysostomos Nicopoulos, Dongkook Park, Reetuparna Das, Yuan Xie, Narayanan Vijaykrishnan, Mazin S. Younis, and Chita R. Das. 2007. A novel dimensionally-decomposed router for on-chip communication in 3D architectures. In *Proceedings of the International Symposium on Computer Architecture (ISCA'07)*. 138–149.
- Michihiro Koibuchi, Akira Funahashi, Akiya Jouraku, and Hideharu Amano. 2001. L-turn routing: An adaptive routing in irregular networks. In *Proceedings of the International Conference on Parallel Processing (ICPP'01)*. 383–392.
- M. Pawan Kumar, Anish S. Kumar, Srinivasan Murali, Luca Benini, and Kamakoti Veezhinathan. 2011. A method for integrating network-on-chip topologies with 3D ICs. In *Proceedings of the IEEE Computer Society Annual Symposium on VLSI (ISVLSI'11)*. 60–65.
- Jinho Lee, Dongwoo Lee, Sunwook Kim, and Kiyoun Choi. 2013a. Deflection routing in 3D network-on-chip with limited vertical bandwidth. *ACM Trans. Des. Autom. Electron. Syst.* 18, 50.
- Jinho Lee, Dongwoo Lee, Sunwook Kim, and Kiyoun Choi. 2013b. Deflection routing in 3D network-on-chip with TSV serialization. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC'13)*. 29–34.
- Feihui Li, Chrysostomos Nicopoulos, Thomas Richardson, Yuan Xie, and Narayanan Vijaykrishnan. 2006. Design and management of 3D chip multiprocessors using network-in-memory. In *Proceedings of the International Symposium on Computer Architecture (ISCA'06)*. 130–141.
- Cheng Liu, Lei Zhang, Yinhe Han, and Xiaowei Li. 2011. Vertical interconnects squeezing in symmetric 3D mesh network-on-chip. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC'11)*. 357–362.
- Gabriel H. Loh. 2008. 3D-stacked memory architectures for multi-core processors. In *Proceedings of the International Symposium on Computer Architecture (ISCA'08)*. 453–464.
- Igor Loi, Federico Angiolini, Shinobu Fujita, Subhasish Mitra, and Luca Benini. 2011. Characterization and implementation of fault-tolerant vertical links for 3D networks-on-chip. *IEEE Trans. Comput.-Aided Des. Integr. Circ. Syst.* 30, 124–134.
- Olav Lysne, Tor Skeie, Sven-Arne Reinemo, and Ingebjørg Theiss. 2006. Layered routing in irregular networks. *IEEE Trans. Parallel Distrib. Syst.* 17, 51–65.
- Hiroki Matsutani, Paul Bogdan, Radu Marculescu, Yasuhiro Take, Daisuke Sasaki, Hao Zhang, Michihiro Koibuchi, Tadahiro Kuroda, and Hideharu Amano. 2013. A case for wireless 3D NoCs for CMPs. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC'13)*. 23–28.
- Andres Mejia, Jose Flich, Jose Duato, Sven-Arne Reinemo, and Tor Skeie. 2006. Segment-based routing: An efficient fault-tolerant routing algorithm for meshes and tori. In *Proceedings of the International Parallel and Distributed Processing Symposium (IPDPS'06)*. 105.
- Jin Ouyang, Jing Xie, Matthew Poremba, and Yuan Xie. 2010. Evaluation of using inductive/capacitive-coupling vertical interconnects in 3D network-on-chip. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD'10)*. 477–482.

- Dongkook Park, Soumya Eachempati, Reetuparna Das, Asit K. Mishra, Yuan Xie, Narayanan Vijaykrishnan, and Chita R. Das. 2008. MIRA: A multi-layered on-chip interconnect router architecture. In *Proceedings of the International Symposium on Computer Architecture (ISCA'08)*. 251–261.
- Sudeep Pasricha. 2009. Exploring serial vertical interconnects for 3D ICs. In *Proceedings of the Design Automation Conference (DAC'09)*. 581–586.
- Sudeep Pasricha. 2012. A framework for TSV serialization-aware synthesis of application specific 3D networks-on-chip. In *Proceedings of the International Conference on VLSI Design (VLSID'12)*. 268–273.
- Amir-Mohammad Rahmani, Khalid Latif, Kameswar Rao Vaddina, Pasi Liljeberg, Juha Plosila, and Hannu Tenhunen. 2011. Congestion aware, fault tolerant, and thermally efficient inter-layer communication scheme for hybrid NoC-bus 3D architectures. In *Proceedings of the International Symposium on Networks-on-Chip (NoCS'11)*. 65–72.
- José Carlos Sancho, Antonio Robles, and José Duato. 2000a. A flexible routing scheme for networks of workstations. In *Proceedings of the International Symposium on High Performance Computing (ISHPC'00)*. 260–267.
- José Carlos Sancho, Antonio Robles, and José Duato. 2000b. A new methodology to compute deadlock free RE-DELF: An energy-efficient deadlock-free routing for 3-D NoCs with partial vertical connections 23routing tables for irregular networks. In *Network-Based Parallel Computing, Communication, Architecture, and Applications*. Springer, 45–60.
- Michael D. Schroeder, Andrew D. Birrell, Michael Burrows, Hal Murray, Roger M. Needham, Thomas L. Rodeheffer, Edwin H. Satterthwaite, and Charles P. Thacker. 1991. Autonet: A high-speed, self configuring local area network using point-to-point links. *IEEE J. Select. Areas Comm.* 9, 1318–1335.
- Ciprian Seiculescu, Srinivasan Murali, Luca Benini, and Giovanni De Micheli. 2009. SunFloor 3D: A tool for networks on chip topology synthesis for 3D systems on chips. In *Proceedings of the Design, Automation, and Test in Europe Conference (DATE'09)*. 9–14.
- Tor Skeie, Olav Lysne, and Ingebjørg Theiss. 2002. Layered shortest path (LASH) routing in irregular system area networks. In *Proceedings of the International Parallel and Distributed Processing Symposium (IPDPS'02)*. 162–169.
- Tor Skeie, Frank Olaf Sem-Jacobsen, Samuel Rodrigo, José Flich, Davide Bertozzi, and Simone Medardoni. 2009. Flexible DOR routing for virtualization of multicore chips. In *Proceedings of the International Symposium on System-on-Chip (SoC'09)*. 73–76.
- Chen Sun, Chia-Hsin Owen Chen, George Kurian, Lan Wei, Jason Miller, Anant Agarwal, Li-Shiuan Peh, and Vladimir Stojanovic. 2012. DSENT-A tool connecting emerging photonics with electronics for optoelectronic networks-on-chip modeling. In *Proceedings of the International Symposium on Networks-on-Chip (NoCS'12)*. 201–210.
- Tilera. 2009. The TILE-GxTM processor family. <http://www.tilera.com/products/processors>.
- Paul Wettin, Jacob Murray, Ryan Kim, Xinmin Yu, Partha Pratim Pande, and Deukhyoun Heo. 2014. Performance evaluation of wireless NoCs in presence of irregular network routing strategies. In *Proceedings of the Design, Automation, and Test in Europe Conference (DATE'14)*. 1–6.
- Steven Cameron Woo, Moriyoshi Ohara, Evan Torrie, Jaswinder Pal Singh, and Anoop Gupta. 1995. The SPLASH-2 programs: Characterization and methodological considerations. In *Proceedings of the International Symposium on Computer Architecture (ISCA'95)*. 24–36.
- Jie Wu and Li Sheng. 1999. Deadlock-free routing in irregular networks using prefix routing. In *Proceedings of the International Symposium on Computer Architecture (ISCA'99)*. 424–430.
- Thomas Canhao Xu, Pasi Liljeberg, and Hannu Tenhunen. 2010. A study of through silicon via impact to 3D network-on-chip design. In *Proceedings of the International Conference on Electronics and Information Engineering (ICEIE'10)*. V1-333–V1-337.
- Yi Xu, Yu Du, Bo Zhao, Xiuyi Zhou, Youtao Zhang, and Jun Yang. 2009. A low-radix and low-diameter 3D interconnection network design. In *Proceedings of the International Symposium on High Performance Computer Architecture (HPCA'09)*. 30–42.
- Shan Yan and Bill Lin. 2011. Design of application-specific 3D networks-on-chip architectures. In *3D Integration for NoC-Based SoC Architectures*. Springer, 167–191.
- Mingyang Zhu, Jinho Lee, and Kiyoun Choi. 2012. An adaptive routing algorithm for 3D mesh NoC with limited vertical bandwidth. In *Proceedings of the International Conference on VLSI and System-on-Chip (VLSI-SoC'12)*. 18–23.

Received October 2014; revised January 2015; accepted March 2015